

MYSQL

Practical & Placement Notes

Simple concepts • Practical SQL • Interview patterns

Focus	Write queries, understand results, and solve unfamiliar SQL problems.
Core	SELECT, filtering, functions, GROUP BY, HAVING, JOINS, subqueries, CASE, views, indexes and transactions.
Placement	Second highest salary, duplicates, top-N, department-wise maximum, missing relationships and date-based queries.

1. Database Basics

Database: An organized place to store data. A table stores data in rows and columns.

Row: One record. Column: One type of information. Primary Key: A column that uniquely identifies each row.

RDBMS: A relational database system where data is stored in related tables. MySQL is an RDBMS.

Foreign Key: A column used to connect one table with another table.

Think of a company database: employees stores employee details, departments stores department details, and department_id connects them.

Create a database and table:

```
CREATE DATABASE company_db;
USE company_db;

CREATE TABLE employees (
  employee_id INT PRIMARY KEY,
  employee_name VARCHAR(100) NOT NULL,
  department_id INT,
  salary DECIMAL(10,2),
  hire_date DATE
);
```

Common data types:

Type	Use
INT	Whole numbers
DECIMAL(10,2)	Money / exact decimal values
VARCHAR(100)	Text
DATE	Date such as 2026-10-01
DATETIME	Date + time
BOOLEAN	True / false style values

Constraints:

- PRIMARY KEY – unique identifier.
- NOT NULL – value is required.
- UNIQUE – duplicate values are not allowed.
- DEFAULT – supplies a value when none is given.
- FOREIGN KEY – maintains a relationship between tables.

2. INSERT, UPDATE, DELETE and SELECT

These commands are used constantly in practical SQL work.

```
INSERT INTO employees
(employee_id, employee_name, department_id, salary, hire_date)
VALUES
(1, 'Rahul', 10, 55000, '2024-06-10'),
(2, 'Priya', 20, 68000, '2023-03-15'),
(3, 'Sameer', 10, 72000, '2022-08-20');
```

```
UPDATE employees
SET salary = 75000
WHERE employee_id = 3;
```

Always check the WHERE condition in UPDATE and DELETE. Without it, multiple/all rows may be affected.

```
DELETE FROM employees
WHERE employee_id = 2;
```

Basic SELECT:

```
SELECT * FROM employees;

SELECT employee_name, salary
FROM employees;
```

Filtering with WHERE:

```
SELECT *
FROM employees
WHERE salary > 60000;

SELECT *
FROM employees
WHERE department_id IN (10, 20);

SELECT *
FROM employees
WHERE salary BETWEEN 50000 AND 80000;

SELECT *
FROM employees
WHERE employee_name LIKE 'P%';
```

Useful operators:

- =, >, <, >=, <=, <> for comparisons.
- AND / OR / NOT for combining conditions.
- IN for multiple possible values.
- BETWEEN for a range.
- LIKE for pattern matching. % means any number of characters.
- IS NULL checks missing values; use IS NULL, not = NULL.

Sorting and limiting:

```
SELECT employee_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

Placement practice:

1. Find employees whose salary is greater than 60,000.
2. Find employees from department 10 or 20.
3. Find the top 3 highest-paid employees.
4. Find names starting with 'A'.

5. Find employees whose salary is between 50,000 and 70,000.

3. Aggregate Functions, GROUP BY and HAVING

Aggregate functions calculate information from multiple rows.

Function	Purpose
COUNT()	Number of rows
SUM()	Total
AVG()	Average
MIN()	Smallest value
MAX()	Largest value

```
SELECT COUNT(*) AS total_employees
FROM employees;
```

```
SELECT AVG(salary) AS average_salary
FROM employees;
```

```
SELECT MAX(salary) AS highest_salary
FROM employees;
```

GROUP BY creates one result for each group.

```
SELECT department_id,
COUNT(*) AS employee_count,
AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id;
```

WHERE vs HAVING: WHERE filters individual rows before grouping. HAVING filters groups after GROUP BY.

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
WHERE salary > 40000
GROUP BY department_id
HAVING AVG(salary) > 60000;
```

Placement practice:

1. Find the number of employees in each department.
2. Find the average salary of each department.
3. Show departments having more than 5 employees.
4. Find the department with the highest average salary.
5. Explain why HAVING is used instead of WHERE for AVG(salary).

4. JOINS

A JOIN combines rows from related tables using a matching condition.

```
CREATE TABLE departments (  
  department_id INT PRIMARY KEY,  
  department_name VARCHAR(100)  
);  
  
SELECT e.employee_name, d.department_name  
FROM employees e  
INNER JOIN departments d  
ON e.department_id = d.department_id;
```

INNER JOIN: returns only matching rows from both tables.

LEFT JOIN: returns every row from the left table, even when there is no match in the right table.

```
SELECT e.employee_name, d.department_name  
FROM employees e  
LEFT JOIN departments d  
ON e.department_id = d.department_id;
```

Find employees who do not have a matching department:

```
SELECT e.employee_name  
FROM employees e  
LEFT JOIN departments d  
ON e.department_id = d.department_id  
WHERE d.department_id IS NULL;
```

Joining more than two tables:

```
SELECT e.employee_name, d.department_name, p.project_name  
FROM employees e  
JOIN departments d  
ON e.department_id = d.department_id  
JOIN projects p  
ON e.employee_id = p.employee_id;
```

Remember: first identify which tables contain the required columns, then identify the relationship column, then choose the JOIN.

Placement practice:

1. Display employee name with department name.
2. Display all departments, including departments with no employees.
3. Find employees without a department.
4. Count employees in each department using JOIN + GROUP BY.
5. Join employees, departments and projects to show project-wise employee counts.

5. Subqueries, CASE and Functions

A subquery is a query written inside another query. It is useful when one query depends on the result of another query.

Example: employees earning more than the average salary:

```
SELECT employee_name, salary
FROM employees
WHERE salary > (
  SELECT AVG(salary)
  FROM employees
);
```

Using IN:

```
SELECT employee_name
FROM employees
WHERE department_id IN (
  SELECT department_id
  FROM departments
  WHERE department_name IN ('IT', 'HR')
);
```

EXISTS checks whether the subquery returns at least one row.

```
SELECT d.department_name
FROM departments d
WHERE EXISTS (
  SELECT 1
  FROM employees e
  WHERE e.department_id = d.department_id
);
```

CASE adds conditional logic to a query.

```
SELECT employee_name, salary,
CASE
  WHEN salary >= 70000 THEN 'High'
  WHEN salary >= 50000 THEN 'Medium'
  ELSE 'Low'
END AS salary_level
FROM employees;
```

Useful functions:

```
SELECT UPPER(employee_name), LOWER(employee_name)
FROM employees;
```

```
SELECT CONCAT(employee_name, ' - ', salary)
FROM employees;
```

```
SELECT LENGTH(employee_name)
FROM employees;
```

```
SELECT ROUND(salary, 0)
FROM employees;
```

```
SELECT YEAR(hire_date), MONTH(hire_date)
FROM employees;
```

6. Placement Query Patterns

These patterns appear frequently in SQL interviews. Learn the logic, not just the query.

1. Second highest salary

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (
  SELECT MAX(salary)
  FROM employees
);
```

2. Find duplicate values

```
SELECT employee_name, COUNT(*) AS total
FROM employees
GROUP BY employee_name
HAVING COUNT(*) > 1;
```

3. Highest salary in each department

```
SELECT department_id, MAX(salary) AS highest_salary
FROM employees
GROUP BY department_id;
```

4. Top 3 salaries

```
SELECT employee_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

5. Employees earning above department average

```
SELECT e.employee_name, e.salary, e.department_id
FROM employees e
WHERE e.salary > (
  SELECT AVG(e2.salary)
  FROM employees e2
  WHERE e2.department_id = e.department_id
);
```

6. Find records with missing relationship

```
SELECT e.*
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id
WHERE d.department_id IS NULL;
```

7. Date-based query

```
SELECT *
FROM employees
WHERE YEAR(hire_date) = 2025;
```

Common interview questions:

- What is the difference between WHERE and HAVING?
- INNER JOIN vs LEFT JOIN?
- What is a primary key and why is it used?
- What is a foreign key?
- What is the difference between DELETE, DROP and TRUNCATE?
- What happens when NULL is used in a comparison?

7. Views, Indexes, Transactions and Normalization

View: A saved SQL query that can be used like a table. It is useful for frequently used reports or simplified access to data.

```
CREATE VIEW employee_report AS
SELECT e.employee_name, d.department_name, e.salary
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id;

SELECT * FROM employee_report;
```

Index: An index helps the database find rows faster, especially when searching or joining on frequently used columns. The trade-off is extra storage and additional work when data changes.

```
CREATE INDEX idx_employee_department
ON employees(department_id);
```

Transaction: A group of operations treated as one unit. COMMIT saves the changes; ROLLBACK cancels uncommitted changes.

```
START TRANSACTION;

UPDATE employees
SET salary = salary + 5000
WHERE department_id = 10;

ROLLBACK; -- cancel the change

-- Or use COMMIT; to save it
```

Normalization: Organizing data to reduce unnecessary duplication and keep related information in appropriate tables.

Example: instead of storing the department name repeatedly for every employee, keep departments in a separate table and store department_id in employees.

Query debugging checklist:

- Check table and column names.
- Check aliases used in SELECT, JOIN and WHERE.
- Check the JOIN condition.
- Check NULL handling.
- Run the query in small parts to find where the problem starts.

8. Practical Placement Set

Use the following questions with the employee/department/project tables. Try solving them without looking at previous examples.

1. Display all employees sorted by salary from highest to lowest.
2. Find the average salary of all employees.
3. Find employees whose salary is greater than the overall average salary.
4. Display employee name and department name.
5. Find departments that have no employees.
6. Find employees who do not belong to any department.
7. Find the number of employees in every department.
8. Display departments having more than 3 employees.
9. Find the highest salary in every department.
10. Find the second highest salary.
11. Find duplicate employee names.
12. Display the top 3 highest-paid employees.
13. Classify employees as High, Medium or Low salary using CASE.
14. Find employees hired in 2025.
15. Find employees earning more than their own department's average salary.
16. Find departments that have at least one employee using EXISTS.
17. Create a view showing employee name, department and salary.
18. Explain the difference between WHERE and HAVING with an example.
19. Explain INNER JOIN and LEFT JOIN with a practical example.
20. Write a query to find products that have never been ordered.

Mini project idea: Online Store

Create four tables: customers, products, orders and order_items. Insert sample data and write queries for customer spending, top products, monthly sales, repeat customers and products never ordered.

9. Quick Revision

Before a placement test, be comfortable with these patterns:

Topic	Remember
SELECT	Choose the columns you need.
WHERE	Filters rows.
ORDER BY	Sorts the result.
LIMIT	Restricts number of rows returned.
GROUP BY	Creates groups for aggregate calculations.
HAVING	Filters grouped results.
JOIN	Combines related tables.
Subquery	Uses the result of one query inside another.
CASE	Adds IF/ELSE-style logic.
IS NULL	Checks for missing values.
VIEW	Stores a reusable query definition.
INDEX	Can speed up searches/joins with storage and write-cost trade-offs.
COMMIT	Saves transaction changes.
ROLLBACK	Cancels uncommitted transaction changes.

Interview habit

When asked to write a query, first identify the required output, then the table(s), then the relationship between tables, and finally the filter/grouping needed. After writing the query, explain it in simple words.

Practice target: solve queries from a fresh schema rather than only repeating the classroom examples.